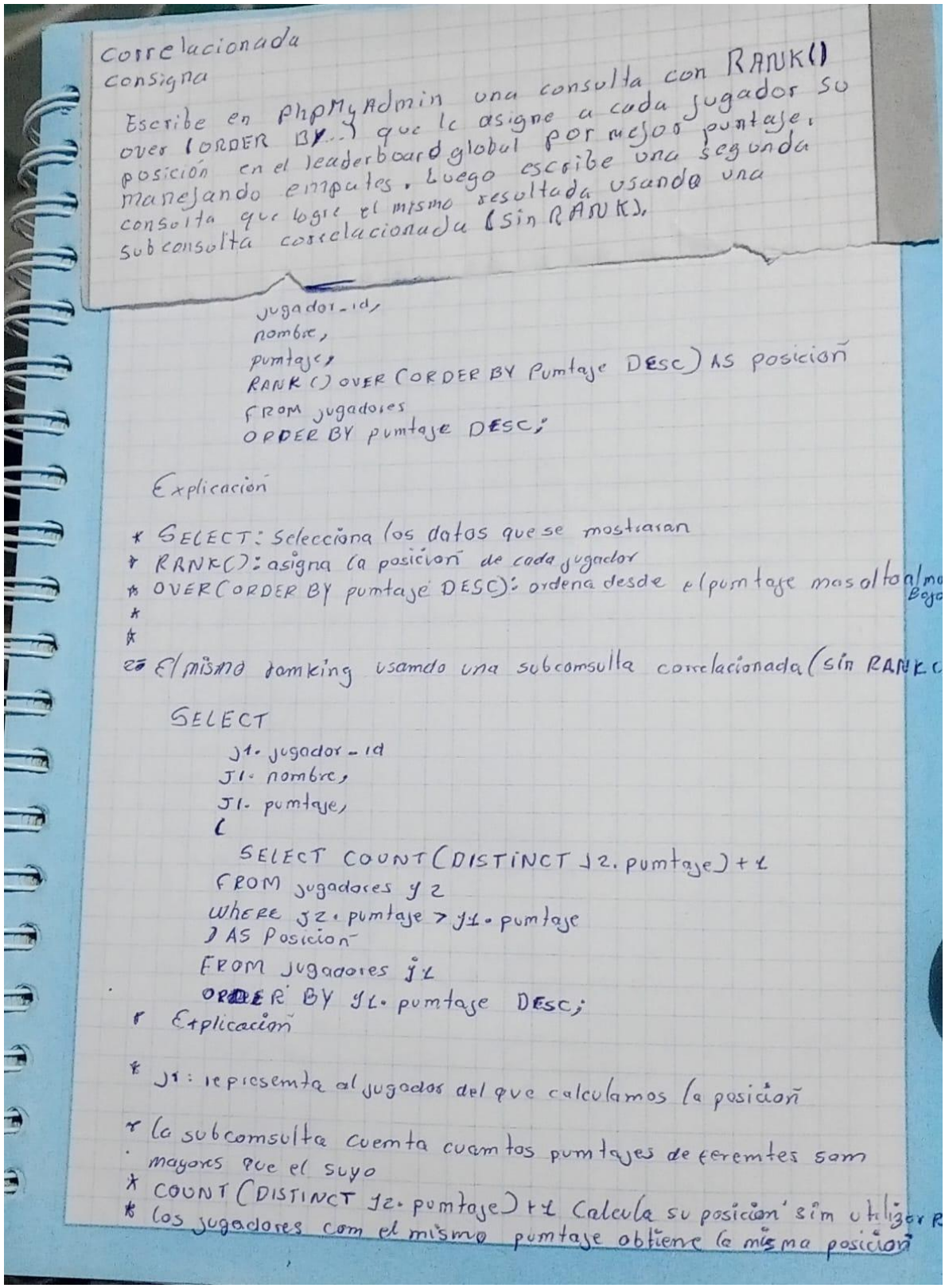
Integrantes
Charli Quispe Mamani
Brayan chambi canaza



Correlacionada
consigna

Escribe en PhpMyAdmin una consulta con RANK() over (ORDER BY...) que le asigne a cada jugador su posición en el leaderboard global por mejor puntaje, manejando empates. Luego escribe una segunda consulta que logre el mismo resultado usando una subconsulta correlacionada (sin RANK).

```
    jugador_id,
    nombre,
    puntaje,
    RANK() OVER (ORDER BY Puntaje DESC) AS posicion
    FROM jugadores
    ORDER BY puntaje DESC;
```

Explicación

* SELECT: Selecciona los datos que se mostraran
* RANK(): asigna la posicion de cada jugador
* OVER(ORDER BY puntaje DESC): ordena desde el puntaje mas alto al mo Bajo
*
*

El mismo ranking usando una subconsulta correlacionada (sin RANK()

```
SELECT
    j1.jugador_id
    j1.nombre,
    j1.puntaje,
    (
      SELECT COUNT(DISTINCT j2.puntaje) + 1
      FROM jugadores j2
      WHERE j2.puntaje > j1.puntaje
    ) AS Posicion
    FROM jugadores j1
    ORDER BY j1.puntaje DESC;
```

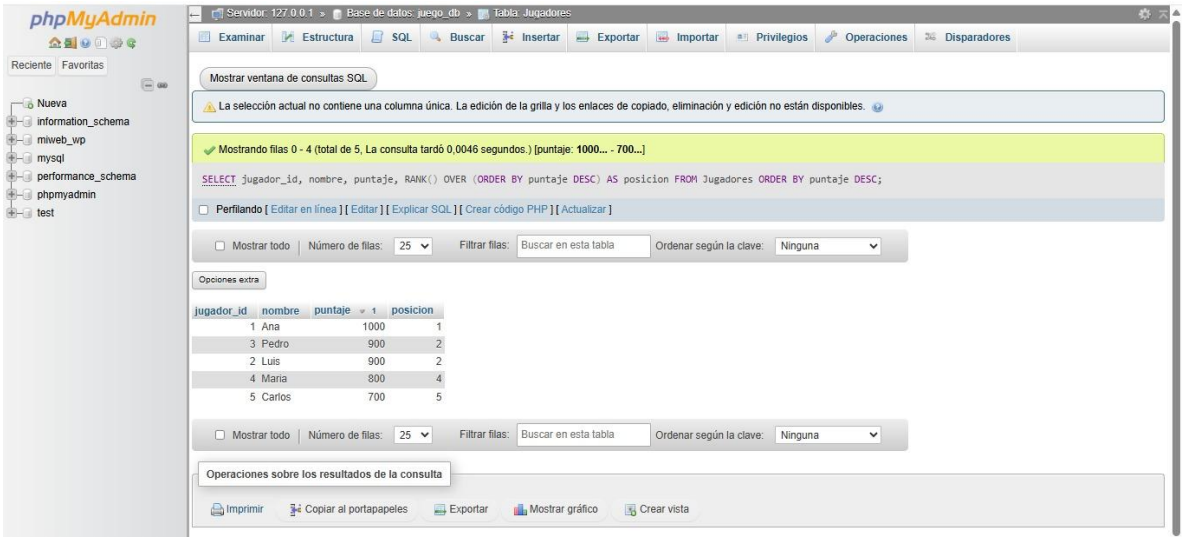
* Explicación

* j1: representa al jugador del que calculamos la posicion
* la subconsulta cuenta cuantos puntajes de diferentes son mayores que el suyo
* COUNT(DISTINCT j2.puntaje) + 1 Calcula su posicion sin utilizar R
* los jugadores con el mismo puntaje obtiene la misma posicion

Integrantes

Charli Quispe Mamani

Brayan chambi canaza



Integrantes
Charli Quispe Mamani
Brayan chambi canaza

Pregunta 2

Escribe una consulta que encuentre todos los jugadores que nunca han obtenido ningún item (No aparecen en inventario_item). usando left join... is NULL y otra equivalente con Not Exists,

1: Consulta usando LEFT JOIN... IS NULL

```
SQL
SELECT
    j.jugador_id,
    j.nombre
FROM jugadores AS j
LEFT JOIN inventario_Items AS i
ON j.jugador_id = i.jugador_id
WHERE i.jugador_id IS NULL
```

Explicación

* jugadores AS j: Selecciona todos los jugadores
* LEFT JOIN: busca si cada jugador tiene registros en inventario_items.
* WHERE i.jugador_id is null: muestra unicamente los jugadores que no tienen ningun items registrado en el inventario

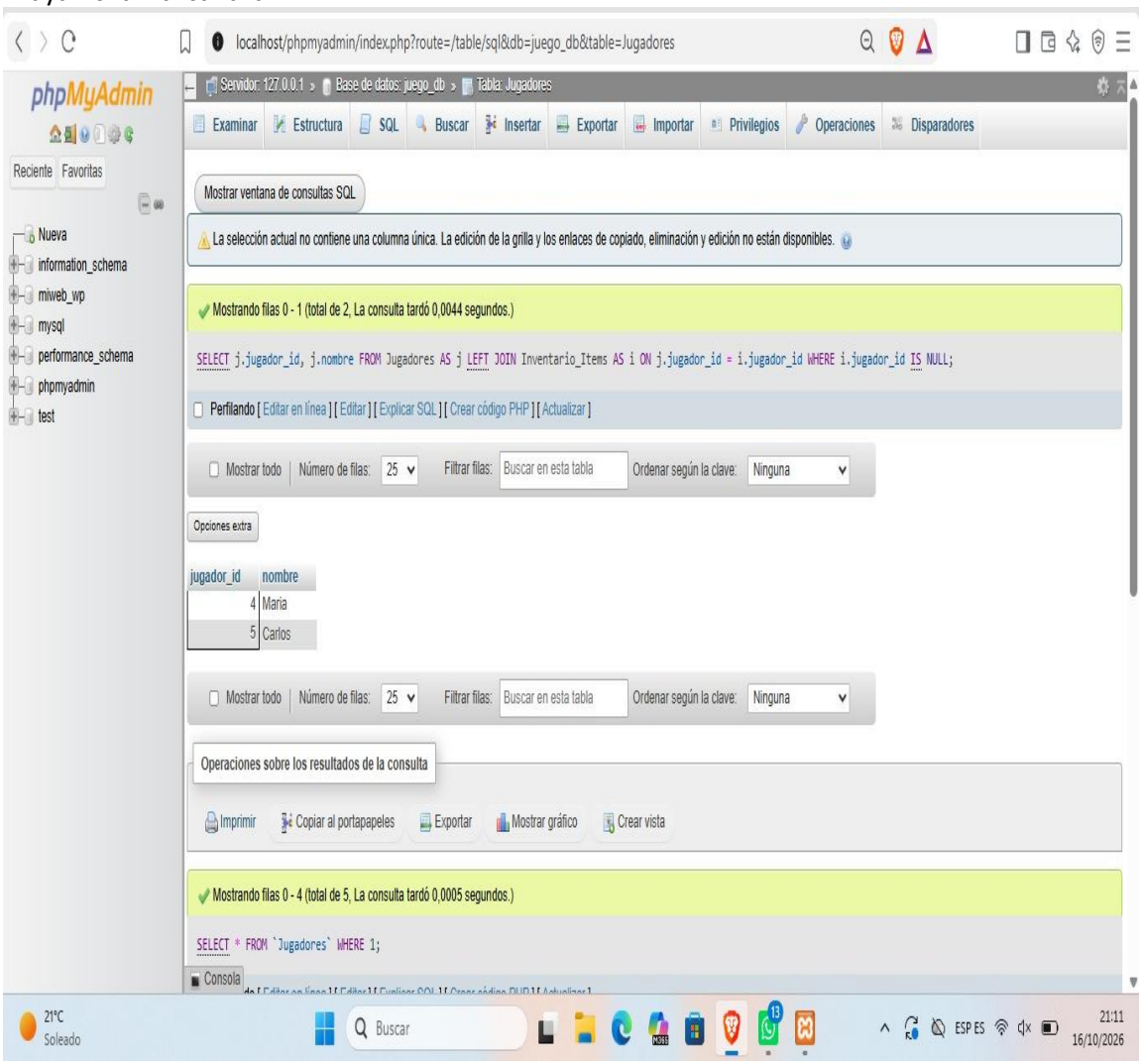
2: Consulta equivalemte usando NO EXISTS

```
SELECT
    j.jugador_id,
    j.nombre
FROM jugadores AS j
WHERE NOT EXISTS (
    SELECT 1
    FROM inventario_items AS i
    WHERE i.jugador_id = j.jugador_id
);
```

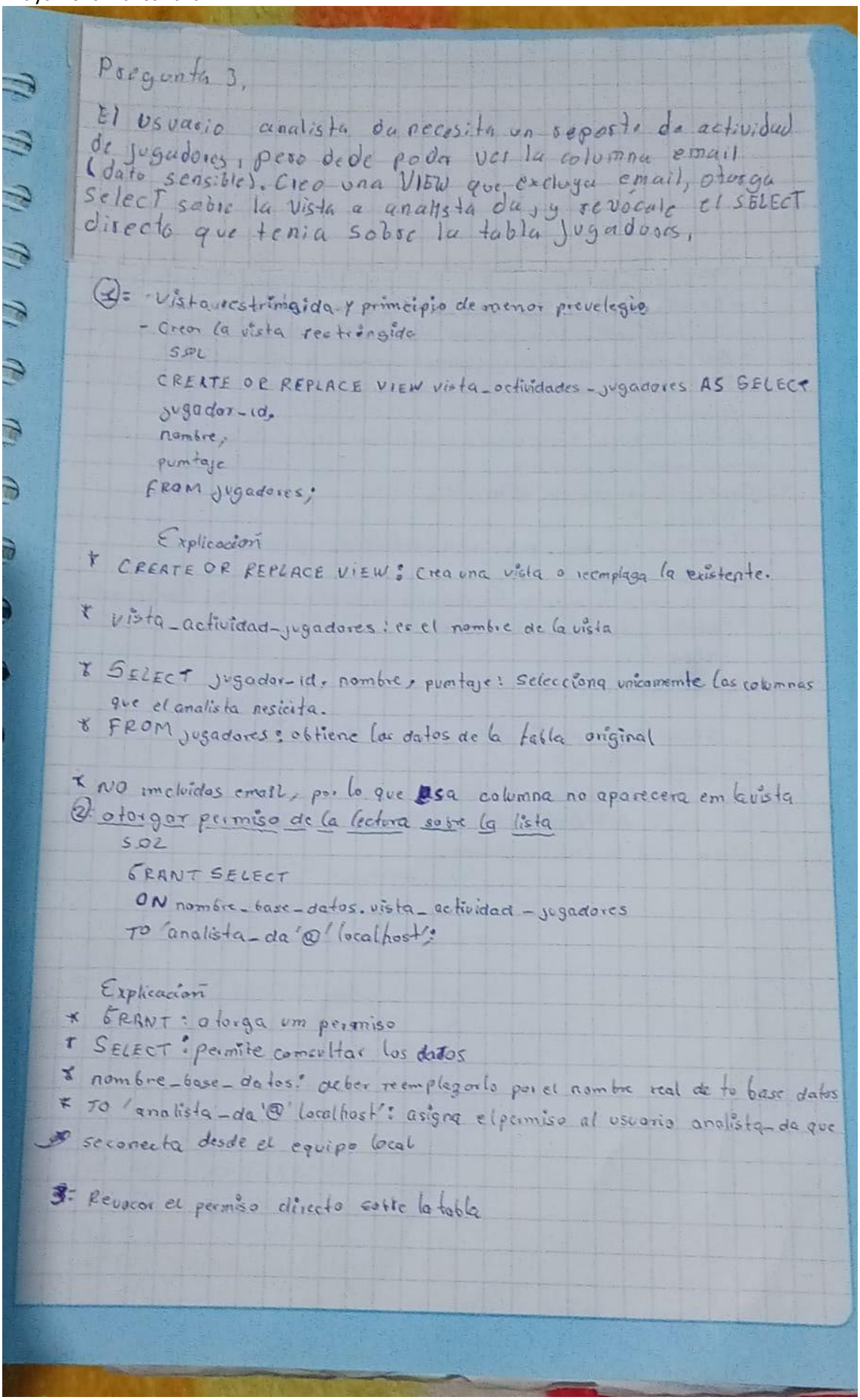
Explicación

* la subconsulta comprueba si el jugador tiene algun item
* NOT EXISTS selecciona los jugadores para los que no existe ningun Registro relacionado en el inventario
* Ambos consultas deben devolver los mismos jugadores

Integrantes
Charli Quispe Mamani
Brayan chambi canaza



Integrantes
Charli Quispe Mamani
Brayan chambi canaza



Pregunta 3,

El usuario analista_da necesita un reporte de actividad de jugadores, pero debe poder ver la columna email (dato sensible). Crea una VIEW que excluya email, otorga SELECT sobre la vista a analista_da, y revoca el SELECT directo que tenía sobre la tabla jugadores.

①: Vista restringida y principio de menor prevelegio
- Crear la vista restringida

SQL

```
CREATE OR REPLACE VIEW vista_actividades_jugadores AS SELECT
jugador_id,
nombre,
puntaje
FROM jugadores;
```

Explicación
* CREATE OR REPLACE VIEW: Crea una vista o reemplaza la existente.
* vista_actividad_jugadores: es el nombre de la vista
* SELECT jugador_id, nombre, puntaje: Selecciona unicamente las columnas que el analista nesicita.
* FROM jugadores: obtiene los datos de la tabla original
* NO incluidos email, por lo que esa columna no aparecera en la vista

②: otorgar permiso de la lectora sobre la lista

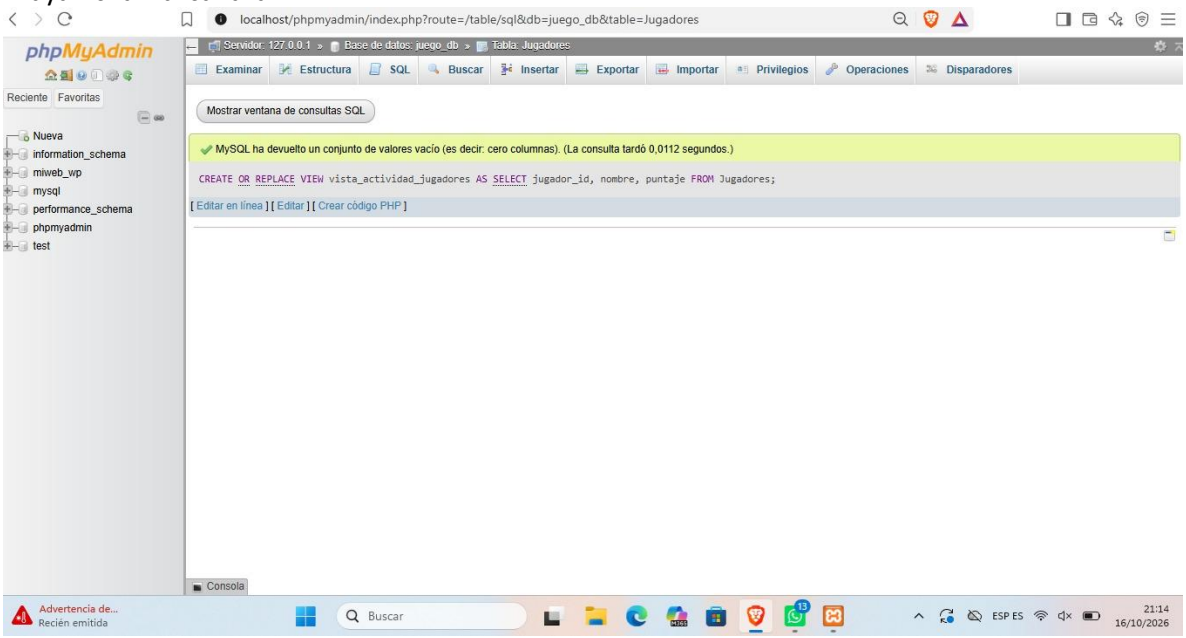
SQL

```
GRANT SELECT
ON nombre_base_datos.vista_actividad_jugadores
TO 'analista_da'@'localhost';
```

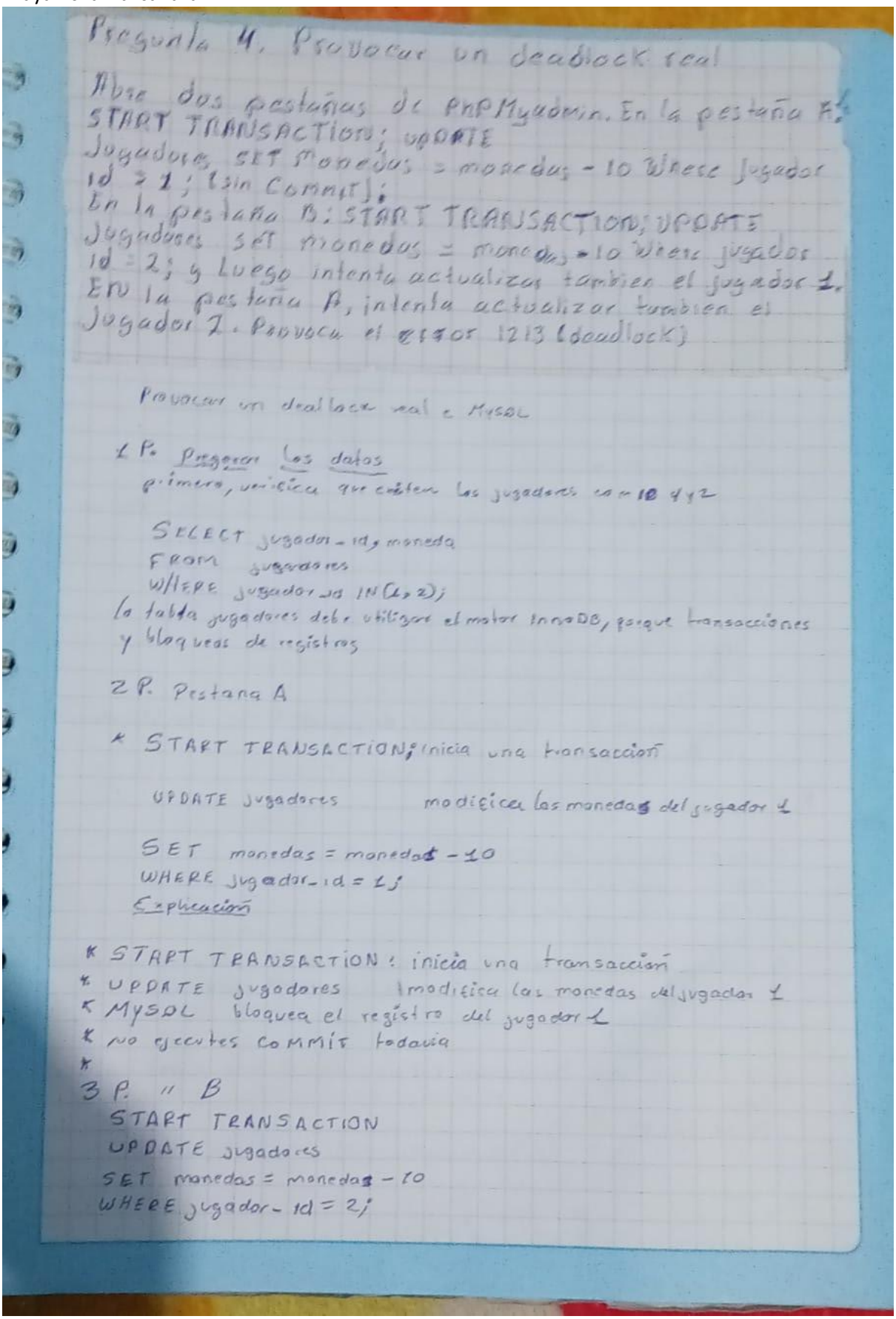
Explicación
* GRANT: otorga un permiso
* SELECT: permite consultar los datos
* nombre_base_datos: deber reemplazarlo por el nombre real de tu base datos
* TO 'analista_da'@'localhost': asigna el permiso al usuario analista_da que se conecta desde el equipo local

③: Revocar el permiso directo sobre la tabla

Integrantes
Charli Quispe Mamani
Brayan chambi canaza



Integrantes
Charli Quispe Mamani
Brayan chambi canaza



Pregunta 4. Provocar un deadlock real

Abre dos pestañas de PhpMyadmin. En la pestaña A: START TRANSACTION; UPDATE Jugadores SET Monedas = monedas - 10 Where Jugador id = 1; (sin Commit);
En la pestaña B: START TRANSACTION; UPDATE Jugadores SET monedas = monedas - 10 Where jugador id = 2; y Luego intenta actualizar tambien el jugador 1.
En la pestaña B, intenta actualizar tambien el Jugador 1. Provoca el error 1213 (deadlock)

Provocar un deadlock real e MySQL

1 P. Preparar los datos
primero, verifica que existen los jugadores con id 1 y 2

```
SELECT jugador_id, moneda
FROM jugadores
WHERE jugador_id IN (1, 2);
```

la tabla jugadores debe utilizar el motor InnoDB, porque transacciones y bloqueos de registros

2 P. Pestaña A

* START TRANSACTION; inicia una transacción

UPDATE jugadores modifica las monedas del jugador 1

```
SET monedas = monedas - 10
WHERE jugador_id = 1;
```

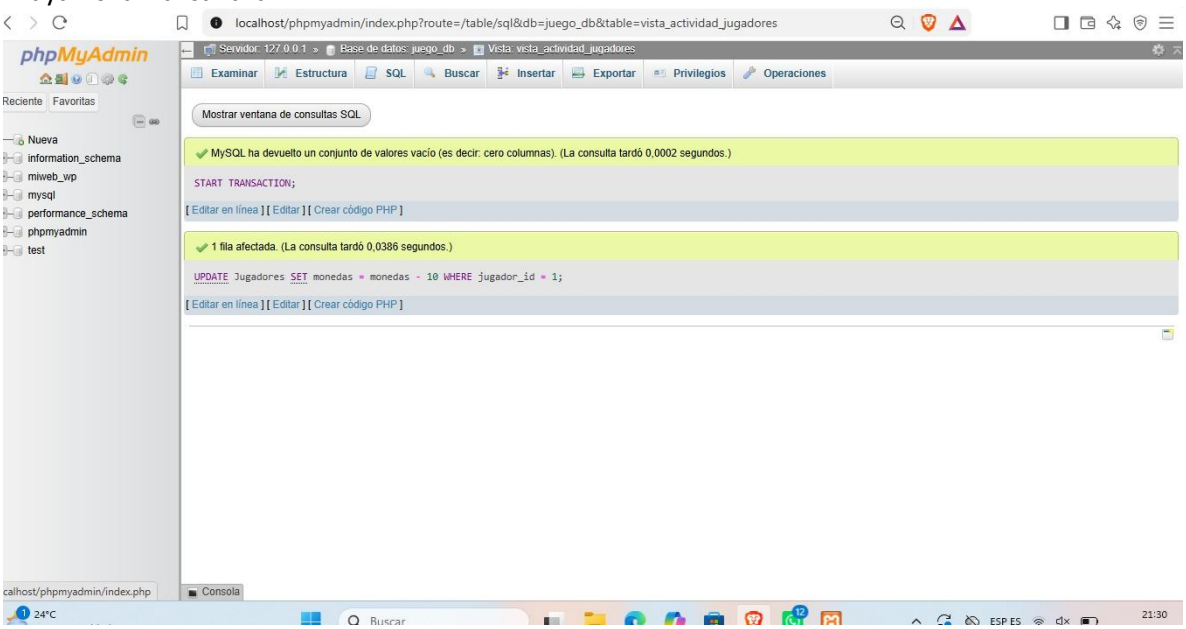
Explicación

* START TRANSACTION: inicia una transacción
* UPDATE jugadores modifica las monedas del jugador 1
* MySQL bloquea el registro del jugador 1
* no ejecutes COMMIT todavia
*

3 P. " B

```
START TRANSACTION
UPDATE jugadores
SET monedas = monedas - 10
WHERE jugador_id = 2;
```

Integrantes
Charli Quispe Mamani
Brayan chambi canaza



Integrantes
Charli Quispe Mamani
Brayan chambi canaza

Pregunta 5. Condición de carrera sin FOR UPDATE

Toma el procedimiento comprar_item_unico y crea una copia modificada que quite la cláusula for UPDATE del SELECT. Ejecuta el mismo Experimento de dos compras simultaneas (Script 3 de la sesión 3) con esta versión insegura.

condición de carrera sin FOR UPDATE

```
/ Procedimiento inseguro sin FOR UPDATE
DELIMITER $$
CREATE PROCEDURE comprar_item_sin_bloqueo(
IN P_id_item INT,
IN P_cantidad INT
)
BEGIN
DECLARE v_stock
INTO v_stock
FROM items
WHERE id_item= p_id_item;
IF v_stock >= P_id_item;
UPDATE items
SET stock=stock- P_cantidad
WHERE id_item = P_id_item;
commit;
SELECT 'compra realizada' AS resultado;

ELSE
ROLLBACK; 'stock insuficiente' AS resultado
SELECT 'stock insuficiente' AS resultado;
END IF;
END$$
DELIMITER;
```

2 P. preparar stock

```
UPDATE items
SET stock = 5
WHERE id_item = 1;
```

3 P. simular dos compras simultaneas

S: A

```
CALL comprar_item_sim_bloqueo(1, 4)
```

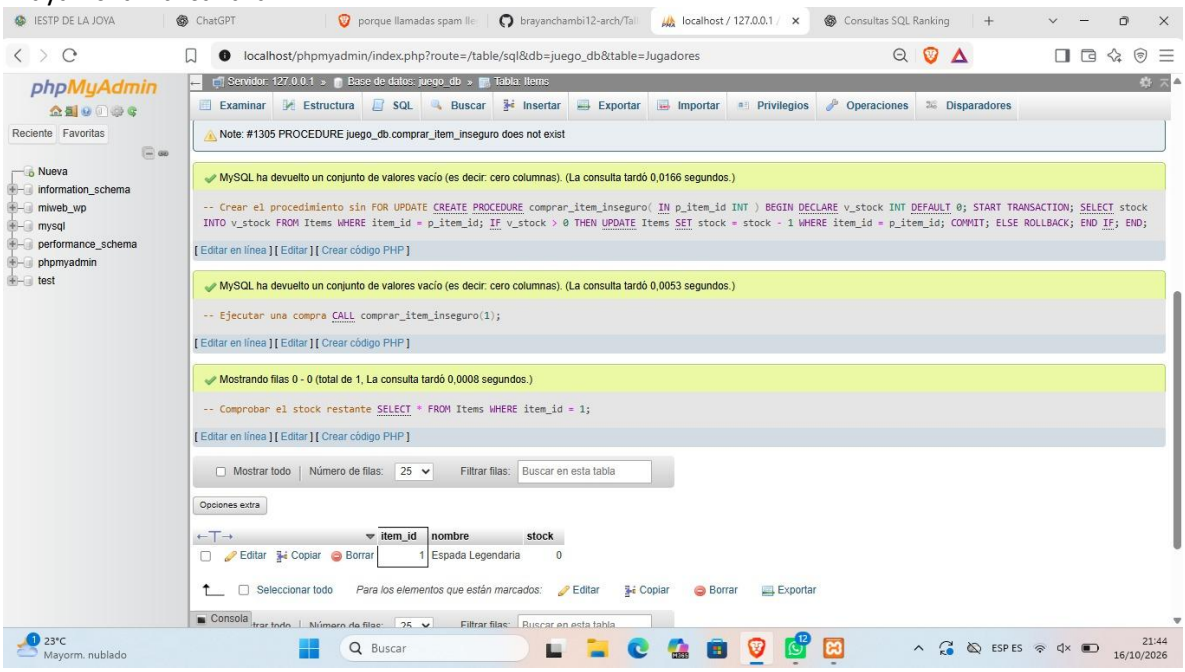
S.B

```
CALL comprar_item_sim_bloqueo
(1, 4);
```

4 P. verificacion el stock

```
SELECT id_item, stock
FROM items
WHERE id_item = 1;
```

Integrantes
Charli Quispe Mamani
Brayan chambi canaza



Integrantes
Charli Quispe Mamani
Brayan chambi canaza

Pregunta 6. Inyección con sentencias apiladas

Usando prepare / EXECUTE para simular, como un backend mal hecho concatenaría texto; construye el payload '; UPDATE jugadores SET monedas = 999999 WHERE nombre_usuario = 'Noob Master'; -- contra una consulta dinámica concatenada, y luego el mismo payload contra buscar_jugador_seguro.

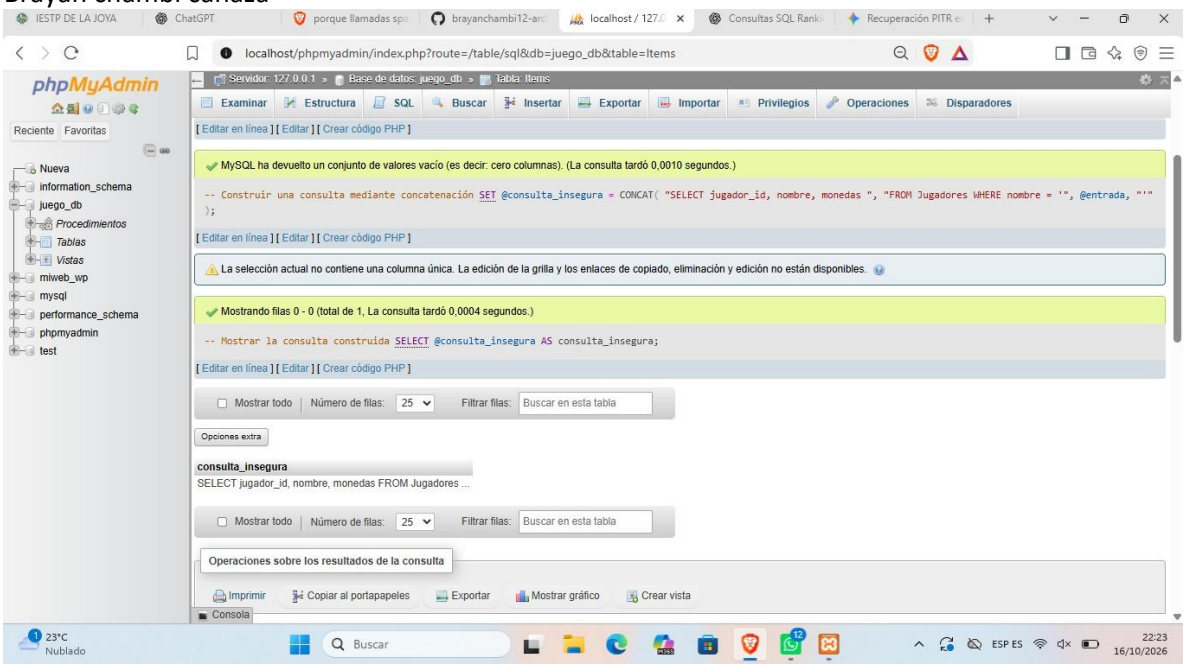
/ codigo para simular la inyección e phpMyadm.
Este ejemplo o ejercicio compara una consulta dinamica

— Inyección SQL con sentencias apiladas

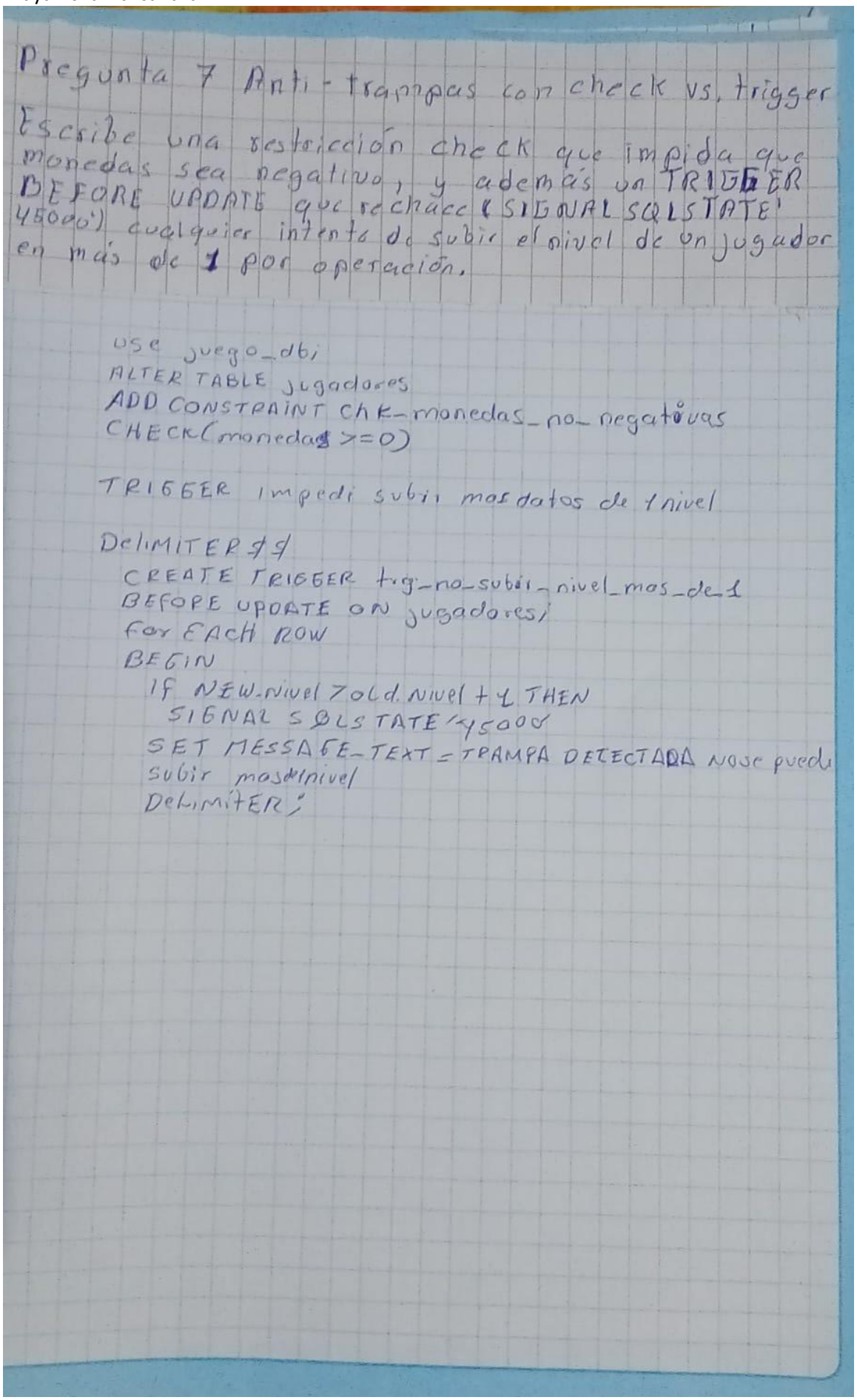
```
use juego_bd;

SET @entrada =
  "'; UPDATE jugadores SET monedas = 999999 WHERE nombre = 'Noob Master'; --";
SET @consulta_insegura = CONCAT(
"SELECT jugador_id, nombre, monedas",
"FROM jugadores WHERE nombre = '",
  @entrada
"', '"
);
  Select @consulta_insegura AS consulta_insegura
  SET @nombre_buscar = 'Noob Master';
  PREPARE buscar_jugador_seguro FROM
  'SELECT jugador_id, nombre, monedas
  FROM jugadores
  WHERE nombre = ?';
  EXECUTE buscar_jugador_seguro USING @nombre_buscar;
  DEALLOCATE PREPARE buscar_jugador_seguro FROM
  'SELECT jugador_id, nombre, monedas
  FROM jugadores
  WHERE nombre = ?';
  EXECUTE buscar_jugador_seguro USING @nombre_buscar;
  DELLOCATE PREPARE buscar_jugador_seguro;
```

Integrantes
Charli Quispe Mamani
Brayan chambi canaza



Integrantes
Charli Quispe Mamani
Brayan chambi canaza



Pregunta 7 Anti-trampas con check vs. trigger

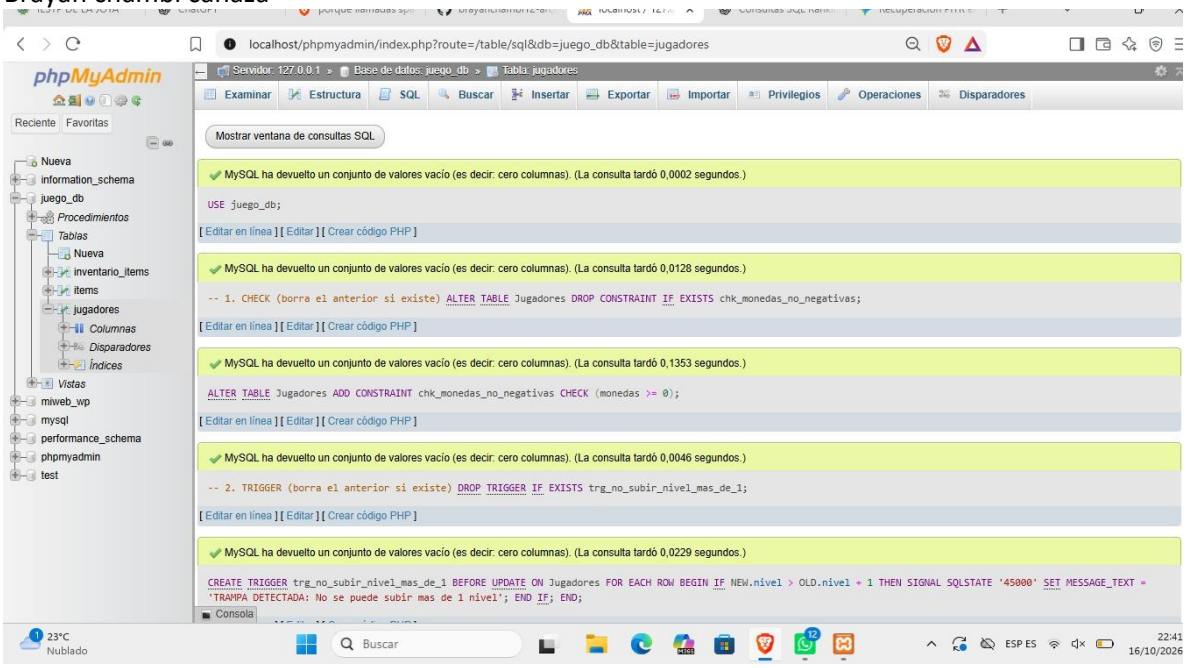
Escribe una restricción check que impida que monedas sea negativo, y además un TRIGGER BEFORE UPDATE que rechace (SIGNAL SQLSTATE '45000') cualquier intento de subir el nivel de un jugador en más de 1 por operación.

```
use juego_db;
ALTER TABLE jugadores
ADD CONSTRAINT chk_monedas_no_negativas
CHECK(monedas >=0)

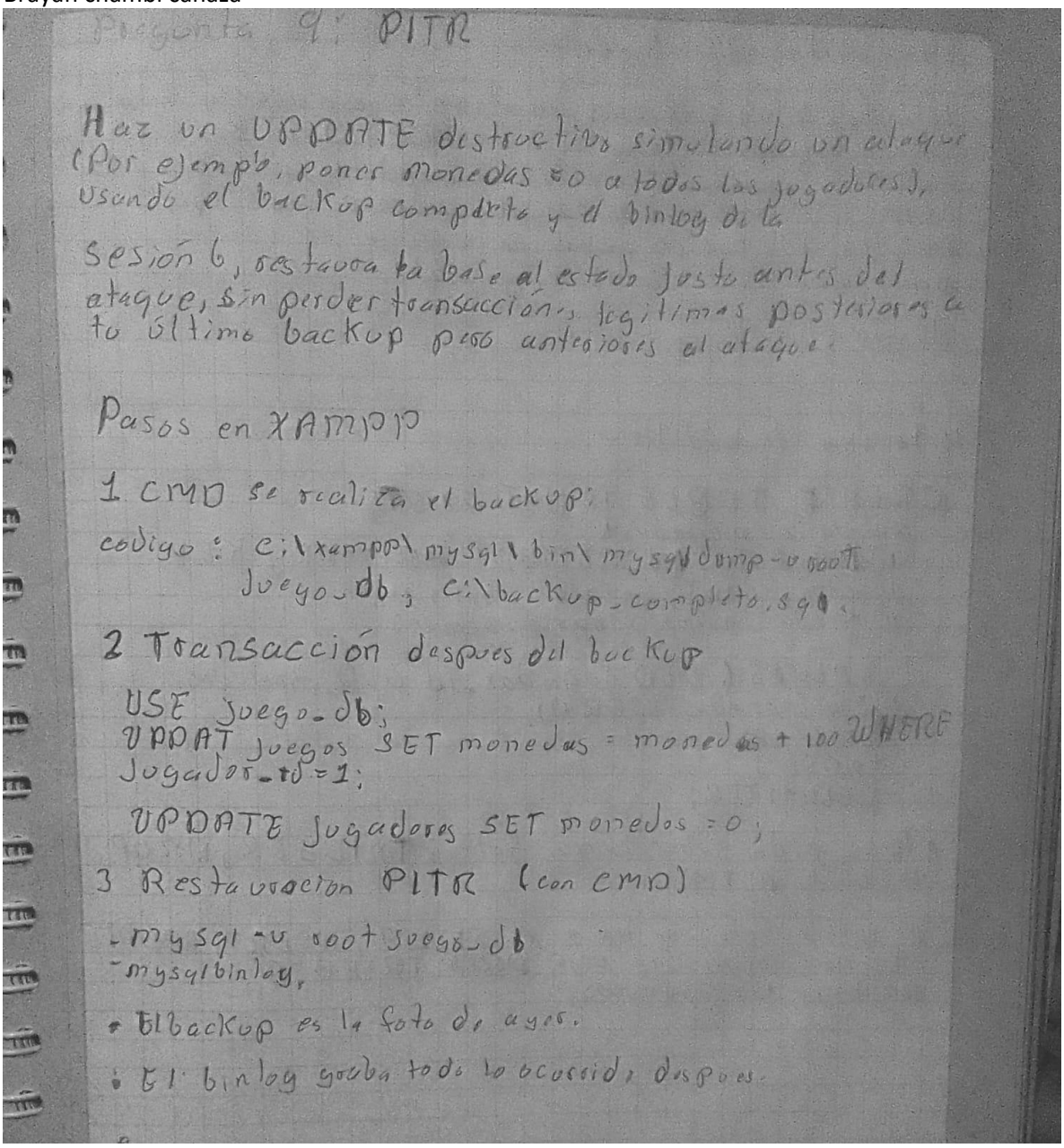
TRIGGER impedi subir mas datos de 1 nivel

DelimITER $$
  CREATE TRIGGER trg_no_subir_nivel_mas_de_1
  BEFORE UPDATE ON jugadores;
  for EACH ROW
  BEGIN
    IF NEW.nivel > old.Nivel + 1 THEN
      SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = TRAMPA DETECTADA NO se puede
    subir mas de nivel
    DelimiTER;
```

Integrantes
Charli Quispe Mamani
Brayan chambi canaza



Integrantes
Charli Quispe Mamani
Brayan chambi canaza



Pregunta 9: PITR

Haz un UPDATE destructivo simulando un ataque (Por ejemplo, poner monedas = 0 a todos los jugadores), usando el backup completo y el binlog de la sesión 6, restaura la base al estado justo antes del ataque, sin perder transacciones legítimas posteriores a tu último backup pero anteriores al ataque.

Pasos en XAMPP

1 CMD se realiza el backup:

codigo: c:\xampp\mysql\bin\mysqldump -u root juego_db > c:\backup_completo.sql;

2 Transacción despues del backup

USE juego_db;
UPDAT juegos SET monedas = monedas + 100 WHERE jugador_id = 1;

UPDATE jugadores SET monedas = 0;

3 Restauracion PITR (con CMD)

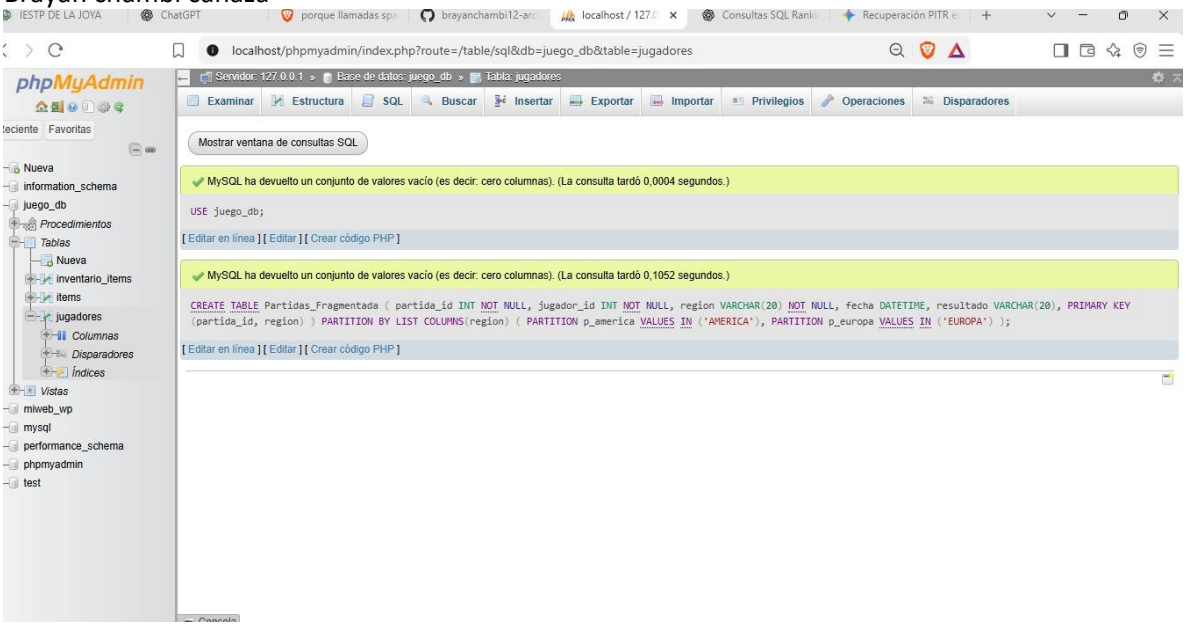
- mysql -u root juego_db
- mysqlbinlog,

- El backup es la foto de ayer.
- El binlog graba todo lo ocurrido despues.

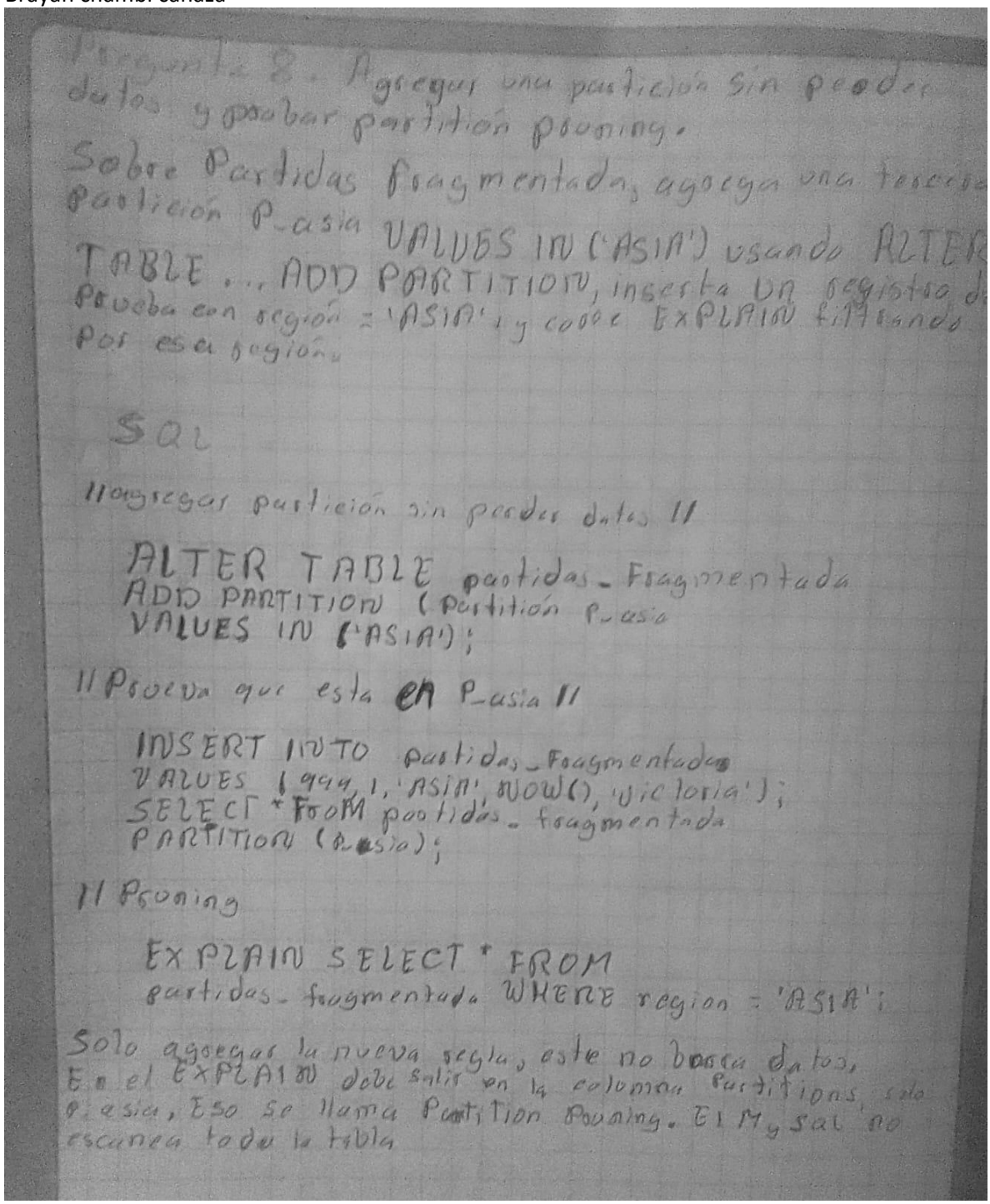
Integrantes

Charli Quispe Mamani

Brayan chambi canaza



Integrantes
Charli Quispe Mamani
Brayan chambi canaza



Pregunta 8. Agregar una particion sin perder datos y probar partition pruning.

Sobre Partidas fragmentada, agrega una tercera particion P_asia VALUES IN ('ASIA') usando ALTER TABLE ... ADD PARTITION, inserta un registro de prueba con region = 'ASIA' y corre EXPLAIN filtrando por esa region.

SQL

```
-- Agregar particion sin perder datos --

ALTER TABLE partidas_fragmentada
ADD PARTITION (Partition P_asia
VALUES IN ('ASIA');

-- Prueba que esta en P_asia --

INSERT INTO partidas_fragmentada
VALUES (999, 1, 'ASIA', NOW(), 'Victoria');
SELECT * FROM partidas_fragmentada
PARTITION (P_asia);

-- Pruning

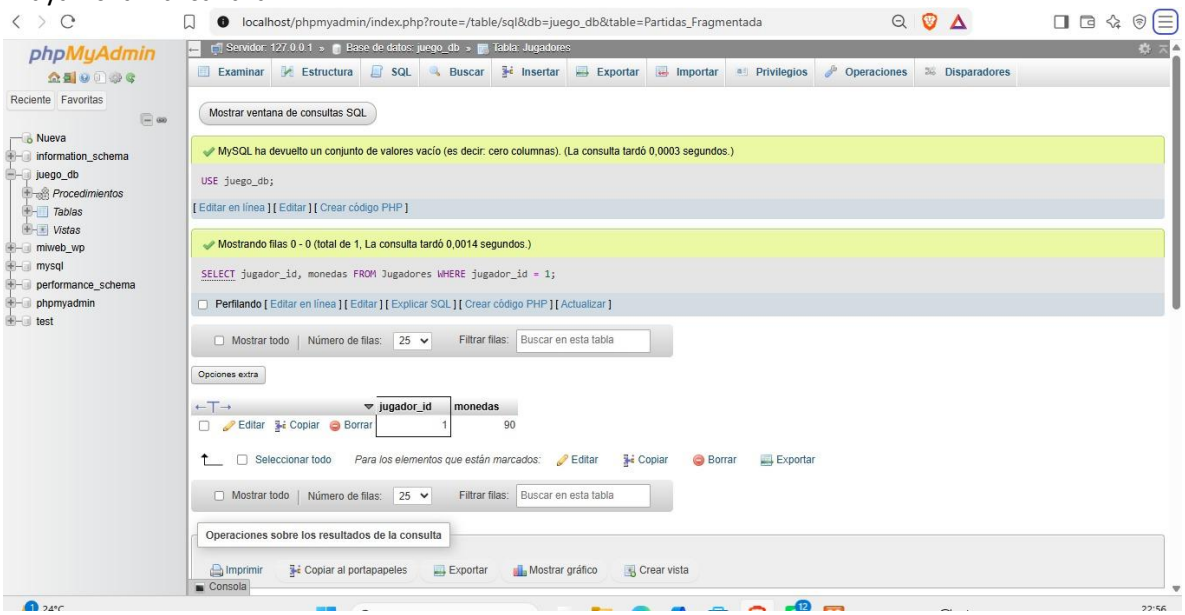
EXPLAIN SELECT * FROM
partidas_fragmentada WHERE region = 'ASIA';
```

Solo agregar la nueva regla, este no borra datos, En el EXPLAIN debe salir en la columna Partitions, solo P_asia, Eso se llama Partition Pruning. El MySql no escanea toda la tabla

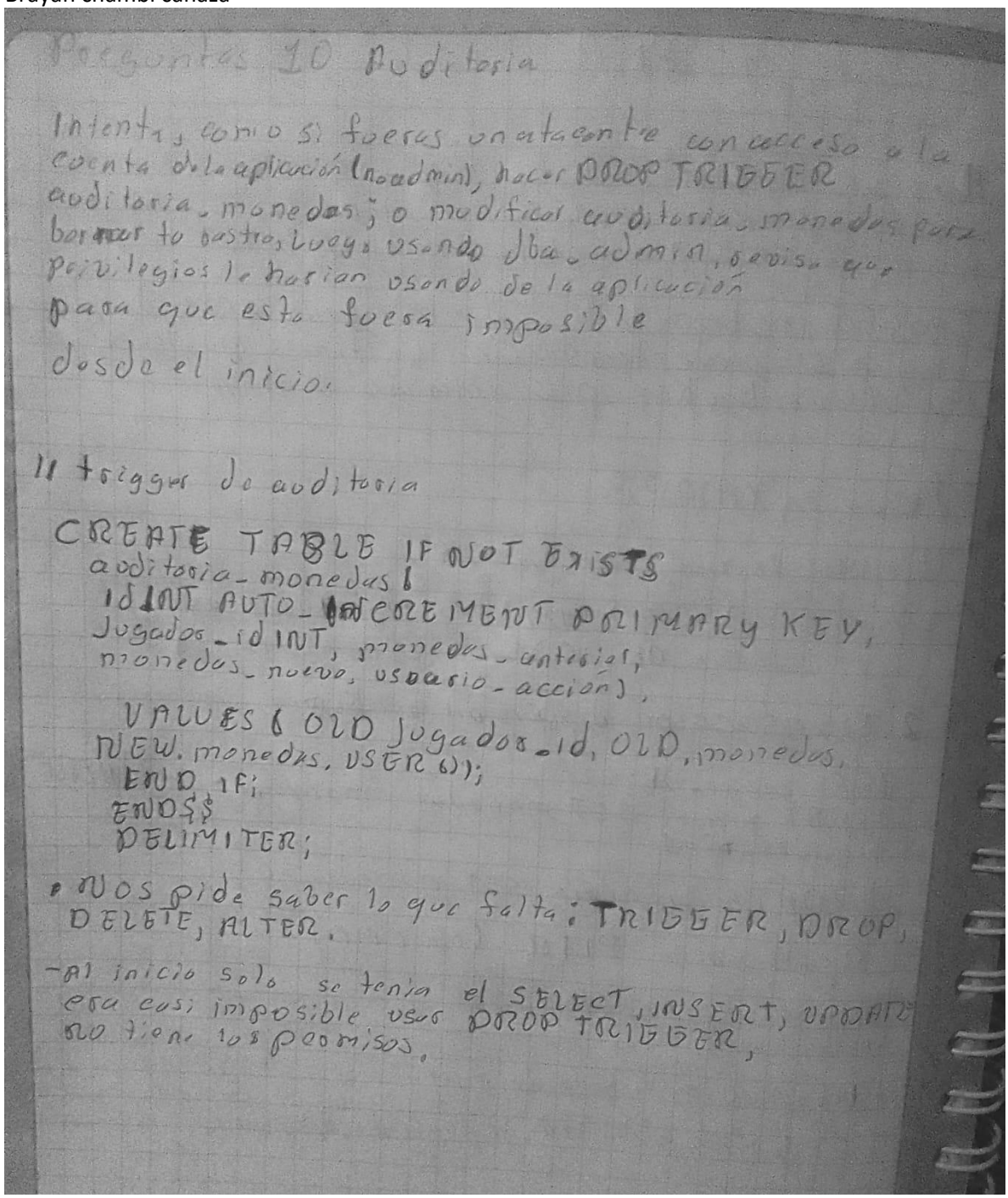
Integrantes

Charli Quispe Mamani

Brayan chambi canaza



Integrantes
Charli Quispe Mamani
Brayan chambi canaza



Preguntas 10 Auditoria

Intenta, como si fueras un atacante con acceso a la cuenta de la aplicación (noadmin), hacer DROP TRIGGER auditoria_monedas; o modificar auditoria_monedas para borrar tu rastro, luego usando dba_admin, revisa que privilegios le harian usando de la aplicación para que esto fuera imposible desde el inicio.

// trigger de auditoria

```
CREATE TABLE IF NOT EXISTS
  auditoria_monedas(
  id INT AUTO_INCREMENT PRIMARY KEY,
  Jugadas_id INT, monedas_anterior,
  monedas_nuevo, usuario_accion),

    VALUES (OLD Jugadas_id, OLD, monedas,
  NEW. monedas, USER());
    END IF;
    END$$
    DELIMITER;
```

- Nos pide saber lo que falta: TRIGGER, DROP, DELETE, ALTER.

-Al inicio solo se tenia el SELECT, INSERT, UPDATE esa casi imposible usar DROP TRIGGER, no tiene los permisos.

Integrantes
Charli Quispe Mamani
Brayan chambi canaza

